Session 2: Integration and Model Selection I

Session Plan

#	Block
1	Reliability: errors, retries, rate limits, request IDs
2	Inputs and outputs: structured outputs, images, documents
3	Enterprise integration: RAG and integration patterns
4	Application interfaces and observability
5	Model selection: capability, latency, cost
6	Reasoning trade-offs
7	Hands-on lab: benchmark one task across models

0. Setup

```
%pip install -q anthropic
```

```
import os, getpass, json, time
if not os.environ.get("ANTHROPIC_API_KEY"):
    os.environ["ANTHROPIC_API_KEY"] = getpass.getpass("Paste your Anthropic API
key: ")

import anthropic
from anthropic import Anthropic

client = Anthropic()
MODEL = "claude-haiku-5-5"

def text_of(message):
    return "".join(b.text for b in message.content if b.type == "text")
```

1. Reliability

Error codes

Code	Type	Retry?	Typical cause
400	`invalid_request_error`	No	Bad parameters, roles don't alternate
401	`authentication_error`	No	Missing or invalid key

Code	Type	Retry?	Typical cause
403	`permission_error`	No	Key not allowed to do this
404	`not_found_error`	No	Wrong or retired model ID
413	`request_too_large`	No	Body too big
429	`rate_limit_error`	**Yes**	Too many requests or tokens
500	`api_error`	**Yes**	Anthropic-side problem
529	`overloaded_error`	**Yes**	API temporarily overloaded

Rule: `4xx` means *you* fix the request (except `429`). `5xx`, `429` and network errors are worth retrying.

Typed exceptions and request IDs

Catch the **specific** class first. Log the **request ID** with every call, because support needs it.

```
try:
    client.messages.create(model="claude-does-not-exist", max_tokens=50,
                           messages=[{"role": "user", "content": "hi"}])
except anthropic.NotFoundError as e:
    print("404 ->", e.message, "| request_id:", e.request_id)
except anthropic.RateLimitError as e:
    print("429 -> wait", e.response.headers.get("retry-after"), "s")
except anthropic.APIStatusError as e:
    print("HTTP error", e.status_code)
except anthropic.APIConnectionError:
    print("Network problem")
```

```
ok = client.messages.create(model=MODEL, max_tokens=200,
                            messages=[{"role": "user", "content": "Say hi."}])
print("request_id:", ok._request_id, "| stop_reason:", ok.stop_reason)
```

Retries and rate limits

- The SDK **already retries** connection errors, `408`, `409`, `429` and `5xx`, **2 times** by default, with exponential backoff. It respects `retry-after`.
- Change it: `Anthropic(max_retries=5, timeout=30.0)`. Turn it off: `max_retries=0`.
- Never retry other `4xx` errors. They fail the same way every time.
- Rate limits are counted per organisation and model: **RPM** (requests), **TPM** (tokens).

Coping with 429: let the SDK retry, queue and cap concurrency, shorten prompts (caching), and move non-urgent work to the **Batch API**.

```
patient = Anthropic(max_retries=5, timeout=30.0)        # whole client
once = client.with_options(max_retries=0)               # one-off, no retries
print(patient.max_retries, once.max_retries)
```

Quiz. A request fails with `400 invalid_request_error`. What should the code do?

```
[( )] Retry with exponential backoff
[(X)] Fix the request and do not retry
[( )] Switch to a bigger model
[( )] Wait for `retry-after`
***

A 400 is a client mistake and fails identically every time.

***
```

2. Inputs and Outputs

Structured outputs

Asking for "JSON only" gives **no guarantee**. Use a schema.

Technique	Guarantee	Current models
Prompt only	None	Yes
Prefill + stop sequence	Format only	**No (400)**
Structured outputs (`messages.parse` / `output_config.format`)	Valid against your schema	Yes
Tool with `strict: true`	Arguments match the schema	Yes

```
from pydantic import BaseModel
from typing import List

class Ticket(BaseModel):
    customer: str
    category: str
    urgent: bool
    products: List[str]

resp = client.messages.parse(
    model=MODEL, max_tokens=1000, output_format=Ticket,
    messages=[{"role": "user", "content":
        "Extract: Priya says her invoice export is broken and she needs it today.
```

```
Products: Billing, Reports."}],
)
print(resp.parsed_output)
```

> Citations cannot be combined with structured outputs (400). Pick one.

Images

Images are content blocks. Cost is about **width x height / 750** tokens. Limits: 100 images per request, 5 MB each.

```
import base64, zlib, struct

def red_square_png(size=64):
    raw = b"".join(b"\x00" + b"\xff\x00\x00" * size for _ in range(size))
    def chunk(t, d):
        c = struct.pack(">I", len(d)) + t + d
        return c + struct.pack(">I", zlib.crc32(t + d) & 0xffffffff)
    return (b"\x89PNG\r\n\x1a\n" + chunk(b"IHDR", struct.pack(">IIBBBBB", size,
size, 8, 2, 0, 0, 0))
            + chunk(b"IDAT", zlib.compress(raw)) + chunk(b"IEND", b""))

img = base64.standard_b64encode(red_square_png()).decode()
r = client.messages.create(model=MODEL, max_tokens=500, messages=[{"role": "user",
"content": [
    {"type": "image", "source": {"type": "base64", "media_type": "image/png",
"data": img}},
    {"type": "text", "text": "What colour is this image? One word."},
]}])
print(text_of(r), "| input_tokens:", r.usage.input_tokens)
```

Documents and citations

PDFs and text use a `document` block. With `citations` on, each claim points back to the source.

```
doc = "Refunds are allowed within 30 days of purchase. Digital goods are non-
refundable once downloaded."
r = client.messages.create(model=MODEL, max_tokens=500, messages=[{"role": "user",
"content": [
    {"type": "document", "title": "Refund policy",
     "source": {"type": "text", "media_type": "text/plain", "data": doc},
     "citations": {"enabled": True}},
    {"type": "text", "text": "Can I get a refund on a downloaded e-book?"},
]}])
for b in r.content:
    if b.type == "text":
        print(b.text, "<-", [c.cited_text for c in (b.citations or [])])
```

Source	Citation type
PDF	`page_location`
Plain text	`char_location`

Files API: upload a big file once (`client.files.upload`), then reference its `file_id`, instead of re-sending bytes each time.

Quiz. You need a response that always matches a fixed JSON schema in production on a current model. Best choice?

```
[( )] Prefill the assistant turn with `{`
[( )] Add "return only JSON" to the prompt
[(X)] Structured outputs with a schema
[( )] Lower `max_tokens`
***

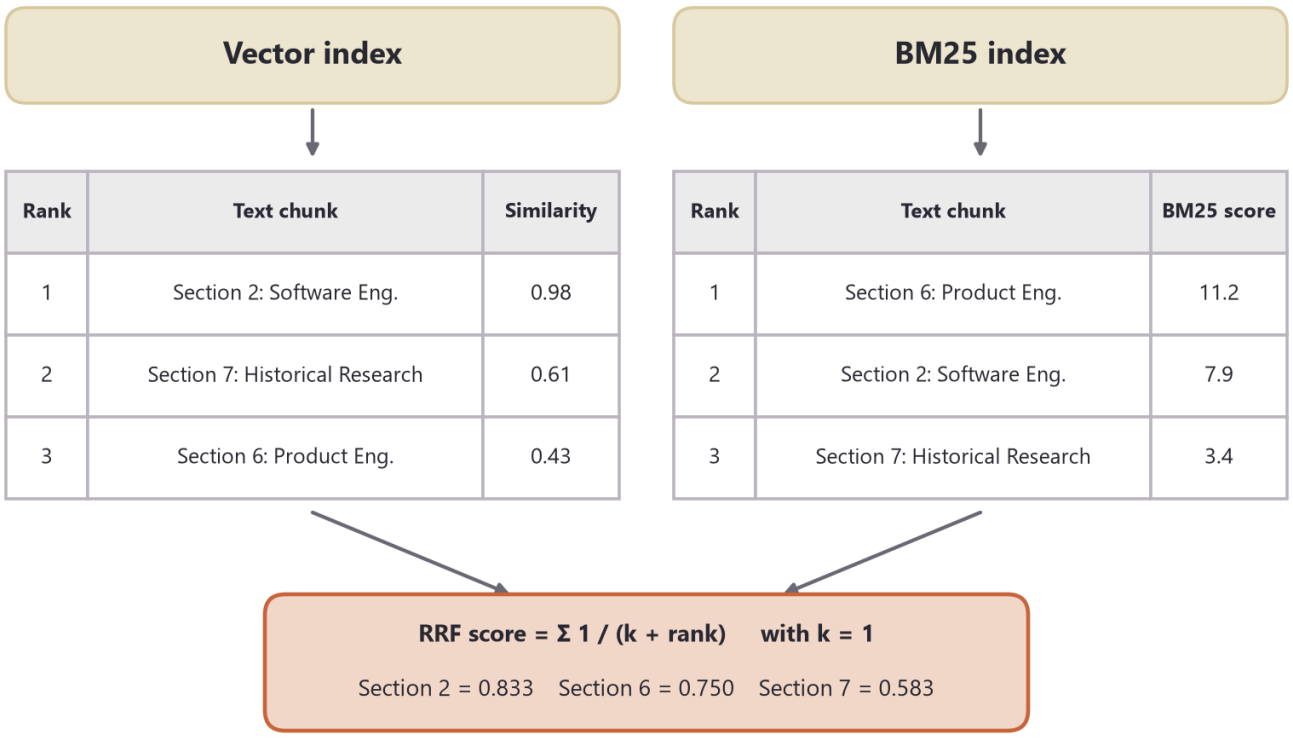
Prefill is rejected on current models, and prompting alone is not a guarantee.

***
```

3. Enterprise Integration: RAG

RAG (retrieval-augmented generation) gives Claude **your** data without retraining. Use it when knowledge is private, large, or changes often.

Step	What happens
1. Chunk	Split documents into small pieces (a few hundred tokens, with overlap)
2. Index	Store chunks with an embedding (vector) and/or a keyword index
3. Retrieve	Find the chunks closest to the question
4. Augment	Put the best chunks in the prompt, **separated from instructions**
5. Generate	Claude answers **from** those chunks, ideally with citations



Search	Strength	Weakness
Semantic (embeddings)	Matches meaning ("car" ~ "automobile")	Can miss exact IDs and names
Lexical (BM25)	Exact terms, codes, names	Misses synonyms
Hybrid	Combine both, then re-rank	More moving parts

Anthropic does not provide an embedding model, so Claude is paired with a separate one. **Voyage AI** is the recommended provider.

Term	Meaning
Embedding	A model turns text into a list of numbers (a **vector**). Texts with similar *meaning* get vectors that point the same way.
Vector search	Embed the question, compare it with every chunk's vector, return the closest chunks
Cosine similarity	How close two vectors point, from -1 to 1. With vectors scaled to length 1, it is just a **dot product**.
Vector database	Stores vectors and does this search fast at scale (pgvector, Pinecone, Chroma). Below, a NumPy array plays that role.

Two rules: embed chunks **once, ahead of time**, and embed the question with the **same model** at query time.

You need a Voyage API key (voyageai.com). The **free tier needs no payment method**, but it is limited to **3 requests per minute**. The code below keeps within that by sending texts in batches, remembering results,

and waiting if the limit is hit. (Adding a payment method raises the limit and keeps the free tokens.)

```
%pip install -q voyageai numpy
```

```
import os, getpass, time
import numpy as np, voyageai

if not os.environ.get("VOYAGE_API_KEY"):
    os.environ["VOYAGE_API_KEY"] = getpass.getpass("Paste your Voyage AI key: ")

vo = voyageai.Client()
EMBED_MODEL = "voyage-3.5"        # check the Voyage docs for the current model
name

chunks = [
    "Employees get 24 days of paid leave per year.",
    "Remote work is allowed up to 3 days a week with manager approval.",
    "Expense claims must be filed within 30 days with receipts.",
    "The VPN is required when accessing internal systems from home.",
]
QUESTIONS = ["Do I need a receipt to get reimbursed?", "Can I work from home?",
"What is the CEO's salary?"]

# The free Voyage tier (no payment method) allows only 3 requests per minute.
# So: send texts in batches, remember results, and wait quietly if the limit is
hit.
_cache = {}

def embed(texts, kind):
    """kind is 'document' for chunks and 'query' for questions."""
    missing = [t for t in dict.fromkeys(texts) if (kind, t) not in _cache]
    if missing:
        for attempt in range(5):
            try:
                result = vo.embed(missing, model=EMBED_MODEL,
input_type=kind).embeddings
                break
            except voyageai.error.RateLimitError:
                print("Voyage free tier: 3 requests/min. Waiting 25 seconds...")
                time.sleep(25)
        else:
            raise RuntimeError("Voyage rate limit: wait a minute and run this cell
again")
        for t, v in zip(missing, result):
            _cache[(kind, t)] = v
    vecs = np.array([_cache[(kind, t)] for t in texts])
    return vecs / np.linalg.norm(vecs, axis=1, keepdims=True)      # length 1

index = embed(chunks, "document")          # done once, ahead of time (1 request)
embed(QUESTIONS, "query")                  # all demo questions in one batch (1
```

```
request)
print("index shape:", index.shape, "(chunks x dimensions)")
print("first 5 numbers of chunk 0:", index[0][:5].round(3))
```

```
def vector_retrieve(question, k=2):
    q = embed([question], "query")[0]
    scores = index @ q                          # cosine similarity with every chunk
    top = np.argsort(-scores)[:k]
    return [(chunks[i], float(scores[i])) for i in top]

def keyword_retrieve(question, k=2):
    q = set(question.lower().split())
    return sorted(chunks, key=lambda c: -len(q & set(c.lower().split())))[:k]

question = QUESTIONS[0]
print("KEYWORD (shared words):")
for c in keyword_retrieve(question):
    print("  ", c)
print("\nVECTOR (shared meaning):")
for c, s in vector_retrieve(question):
    print(f"   {s:.3f}  {c}")
```

Keyword search matched on shared *words* and returned the wrong chunks. Vector search matched on *meaning* ("reimbursed" ~ "expense claims"). The last question has no answer in the documents, so Claude should say it doesn't know. A **similarity threshold** (ignore chunks below a score) is a common extra guard.

```
def rag_answer(question):
    context = "\n".join(f"<doc>{c}</doc>" for c, _ in vector_retrieve(question))
    prompt = (f"<documents>\n{context}\n</documents>\n\n"
              f"Answer using ONLY the documents. If the answer is not there, say
you don't know.\n\n"
              f"Question: {question}")
    return text_of(client.messages.create(model=MODEL, max_tokens=500,
                                          messages=[{"role": "user", "content":
prompt}]))

for q in QUESTIONS:
    print("Q:", q, "\nA:", rag_answer(q), "\n")
```

Integration patterns: RAG for knowledge, **tools or MCP** for live data and actions (Session 4), the **Batch API** for bulk offline jobs, and a **human approval** step before anything irreversible.

Quiz. Claude gives outdated answers about your internal policies. Best fix?

```
[( )] Increase `max_tokens`
[(X)] Retrieve the current policy text and include it in the prompt
[( )] Use a higher temperature
```

```
[( )] Add "be accurate" to the system prompt
***

RAG supplies current, private facts at request time.

***
```

4. Application Interfaces and Observability

Surface	Who uses it	Typical use
claude.ai (web, mobile)	End users	Chat, projects, file analysis
Claude Desktop	End users	Chat plus local MCP servers
Claude Code	Developers	Agentic coding in the terminal or IDE
API / Console	Developers	Build your own product; manage keys and limits
SDKs (Python, TypeScript, ...)	Developers	Typed API access, retries, streaming
Cloud platforms (Bedrock, Vertex AI)	Enterprises	Same models inside your cloud account

SDK configuration: API key (env var), `timeout`, `max_retries`, and the model ID. Pin model IDs in config, not scattered in code.

What to measure on every call

Metric	Why
`request_id`	Support and debugging
model, `stop_reason`	Catch truncation and wrong routing
input / output tokens	Cost tracking
latency	User experience and timeouts
error type and status	Retry vs fix

```
def logged_call(**kwargs):
    start = time.time()
    try:
        m = client.messages.create(**kwargs)
        print(json.dumps({"id": m._request_id, "model": m.model, "stop":
m.stop_reason,
                          "in": m.usage.input_tokens, "out":
m.usage.output_tokens,
                          "sec": round(time.time() - start, 2)}))
        return m
    except anthropic.APIStatusError as e:
        print(json.dumps({"error": e.status_code, "id": e.request_id}))
        raise
```

```
_ = logged_call(model=MODEL, max_tokens=300, messages=[{"role": "user", "content":
"Name a colour."}])
```

5. Model Selection

Prices are USD per million tokens, cached from the docs on 2026-10-06. **Confirm before quoting.**

Model	ID	Input	Output	Positioning
Fable 5.1	`claude-fable-5-1`	10.00	50.00	Most capable, hardest reasoning
Opus 5.5	`claude-opus-5-5`	4.00	20.00	Strong all-rounder for complex work
Sonnet 5.5	`claude-sonnet-5-5`	2.00	10.00	Speed and capability, everyday default
Haiku 5.5	`claude-haiku-5-5`	0.10	0.50	Fastest and cheapest, high volume

Three dials: capability, latency, cost. Output tokens cost several times more than input.

A process that works

1. Define the task and a **pass mark** ("95% of tickets correct").
2. Build a small **test set** with known answers.
3. Start with the **cheapest** model and measure.
4. Move up only if it misses the pass mark.
5. Compare **cost per correct result**, not per call.

Other levers: better prompts, `max_tokens`, prompt caching, Batch API (50% off), effort setting, routing easy work to small models.

```
PRICES = {"claude-haiku-5-5": (0.10, 0.50), "claude-sonnet-5-5": (2.00, 10.00),
          "claude-opus-5-5": (4.00, 20.00), "claude-fable-5-1": (10.00, 50.00)}

def monthly_cost(model, in_tok, out_tok, calls):
    pin, pout = PRICES[model]
    return calls * (in_tok * pin + out_tok * pout) / 1_000_000

for m in PRICES:
    print(f"{m:20s} ${monthly_cost(m, 2000, 500, 100_000):>9,.2f} / month")
```

Quiz. A 95% accuracy target is met by the smallest model on your test set. What do you do?

```
[( )] Use the largest model anyway, to be safe
[(X)] Ship the smallest model, and re-test when models change
[( )] Skip testing, because model names show quality
[( )] Lower the pass mark
***
```

```
Pick the cheapest model that meets the bar, based on measurement.

***
```

6. Reasoning Trade-offs

With **thinking** on, Claude works through the problem before answering. The reply then has a `thinking` block **before** the `text` block.

Benefit	Cost
Better on hard, multi-step problems	More output tokens (billed)
Reasoning you can inspect	Slower first answer

Rule: run your eval **without** thinking first. Turn it on only if accuracy is still short.

How it is set on current models: thinking is **adaptive** (on by default). Control it with `output_config={"effort": ...}` (`low` to `max`). `budget_tokens` is for older models and returns a **400** on current ones. Thinking tokens count toward `max_tokens`.

```
Q = [{"role": "user", "content": "A train leaves at 9:40 and arrives at 14:15. How
long is the trip?"}]
for effort in ["low", "high"]:
    t = time.time()
    m = client.messages.create(model="claude-sonnet-5-5", max_tokens=4000,
                               output_config={"effort": effort}, messages=Q)
    print(f"{effort:5s} out_tokens={m.usage.output_tokens:4d} time={time.time()-
t:4.1f}s  {text_of(m)[:60]!r}")
```

Quiz. Which task most justifies extended thinking?

```
[( )] Classifying a ticket as billing or technical
[( )] Extracting a name from a sentence
[(X)] Planning a multi-step migration with many constraints
[( )] Answering a short factual lookup
***

Thinking pays off on hard multi-step problems, not simple tasks.

***
```

7. Hands-On Lab: Benchmark One Task Across Models

Task: classify support tickets. Measure accuracy, latency and cost for two models.

```
TICKETS = [
    ("My card was charged twice this month", "billing"),
    ("The app crashes when I open settings", "technical"),
    ("How do I change my invoice address?", "billing"),
    ("Login page shows a blank screen", "technical"),
    ("I was overcharged on my last bill", "billing"),
    ("Export to CSV gives an error", "technical"),
]

def classify(model, text):
    t = time.time()
    m = client.messages.create(
        model=model, max_tokens=1000,
        system="Classify the support ticket. Reply with exactly one word: billing
or technical.",
        messages=[{"role": "user", "content": text}],
    )
    return text_of(m).strip().lower(), time.time() - t, m.usage

def benchmark(model):
    correct, secs, cost = 0, 0.0, 0.0
    for text, label in TICKETS:
        guess, dt, u = classify(model, text)
        correct += guess.startswith(label)
        secs += dt
        pin, pout = PRICES[model]
        cost += (u.input_tokens * pin + u.output_tokens * pout) / 1_000_000
    return correct, secs / len(TICKETS), cost

print(f"{'model':20s} {'correct':>8s} {'avg sec':>8s} {'cost $':>10s}")
for model in ["claude-haiku-5-5", "claude-sonnet-5-5"]:
    c, s, cost = benchmark(model)
    print(f"{model:20s} {c:>5d}/{len(TICKETS)} {s:>8.2f} {cost:>10.5f}")
```

Tasks

- [ ] Run it. Which model meets a 100% pass mark at the lowest cost?
- [ ] Add two harder, ambiguous tickets. Does the winner change?
- [ ] Add `claude-opus-5-5`. Is the extra cost justified?
- [ ] Compute **cost per correct result** for each model.

Wrap-Up

Topic	Remember
Reliability	Retry `429` and `5xx` only. SDK retries 2 times. Log `request_id`.
Outputs	Structured outputs for guaranteed schema. Prefill is gone on current models.
Inputs	Images and documents are content blocks. Citations cannot combine with structured outputs.

Topic	Remember
RAG	Chunk, retrieve, augment, answer from the chunks. Hybrid search beats either alone.
Models	Measure on your task. Cheapest model that passes. Cost per correct result.
Thinking	Off first. Effort is the dial. More thinking is more tokens and latency.

Next: Session 3: caching, batching, prompt structure, evaluations.